

DS November Assignment

Natya Vidhan Biswas

B.Sc. (H) C.S. 25771

Write a program to implement insert and search operation in AVL trees.

```
#include <iostream>
using namespace std;

template <typename T>
class Node {
    T val;
    Node* left;
    Node* right;
    int height;
public:
    Node(T v): val(v), left(0), right(0), height(1) {};
    template <typename u>
    friend class AVLTree;
};

template <typename T>
class AVLTree {
    Node<T> *root;
public:
    AVLTree(): root(nullptr) {};

    void insert(T val) {
        root = insertHelper(root, val);
    }

    Node<T>* left_rotate(Node<T>* x) {
        Node<T>* y = x->right;
        Node<T>* T2 = y->left;
        y->left = x;
        x->right = T2;
        x->height = max(height(x->left), height(x->right)) + 1;
        y->height = max(height(y->left), height(y->right)) + 1;

        return y;
    }

    Node<T>* right_rotate(Node<T>* y) {
```

```

Node<T>* x = y->left;
Node<T>* T2 = x->right;

x->right = y;
y->left = T2;
y->height = max(height(y->left), height(y->right)) + 1;
x->height = max(height(x->left), height(x->right)) + 1;
return x;
}

int height(Node<T>* N) {
    if (N == nullptr)
        return 0;
    return N->height;
}

int getBalance(Node<T>* N) {
    if (N == nullptr)
        return 0;
    return height(N->left) - height(N->right);
}

bool search(T val) {
    Node<T>* node = root;
    while (node != nullptr) {
        if (val == node->val)
            return true;
        else if (val < node->val)
            node = node->left;
        else
            node = node->right;
    }
    return false;
}

private:
Node<T>* insertHelper(Node<T>* node, T val) {
    if (node == nullptr)
        return new Node<T>(val);

```

```

        if (val < node->val)
            node->left = insertHelper(node->left, val);
        else
            node->right = insertHelper(node->right, val);

        node->height = 1 + max(height(node->left),
height(node->right));

        int balance = getBalance(node);

        if (balance > 1 && val < node->left->val)
            return right_rotate(node);

        if (balance < -1 && val > node->right->val)
            return left_rotate(node);

        if (balance > 1 && val > node->left->val) {
            node->left = left_rotate(node->left);
            return right_rotate(node);
        }

        if (balance < -1 && val < node->right->val) {
            node->right = right_rotate(node->right);
            return left_rotate(node);
        }

        return node;
    }
};

int main() {
    int nums[8] = {10, 20, 30, 40, 4, 25, 2, 15};
    AVLTree<int> tree;
    for (int num : nums) {
        tree.insert(num);
    }
    tree.search(25);
}

```